

Preguntas

Responde las siguientes preguntas en un documento o en comentarios al final del archivo principal:

1. ¿Cuál es la función de la clase NodeTree?
Aquí los elementos del árbol estarán representados:
`self.value = value`
`self.left = None`
`self.right = None`
2. ¿Qué representa el atributo root?
Representa la raíz del árbol
3. ¿Qué significa que un nodo tenga como hijo el valor None?
Indica que no tiene ningún elemento asignado
4. ¿Cuál es el caso base del método `_insertar`?
`if node is None:`
`return NodeTree(value)`
Cuando se encuentra una posición vacía, se crea y se devuelve un nuevo nodo.
5. ¿Qué sucede cuando `_insertar` encuentra una posición vacía?
Se crea y se devuelve un nuevo Nodo
6. ¿Por qué se utiliza `return node` al final del método `_insertar`?
Porque permite conservar las conexiones entre los nodos del árbol
7. ¿Qué sucede cuando se intenta insertar un valor repetido?
No se crea un nodo duplicado
`else:`
`print("El valor ya existe en el árbol.")`
8. ¿Cómo decide el método `buscar` si debe continuar a la izquierda o a la derecha?
Si el valor es menor, la búsqueda continúa en el lado izquierdo:
`if value < node.value:`
`return self._buscar(node.left, value)`
Si el valor es mayor, continúa en el lado derecho:
`return self._buscar(node.right, value)`
9. ¿Cuál es el caso base de la búsqueda?
La búsqueda no necesita revisar todos los nodos porque utiliza el orden del árbol para decidir hacia qué lado continuar.

10. ¿Cuál es el caso base de los recorridos?

Cuando el método llega a una posición vacía, deja de realizar llamadas.

11. ¿En qué momento se muestra la raíz en el recorrido preorden?

Raíz → Izquierda → Derecha

12. ¿En qué momento se muestra la raíz en el recorrido inorden?

Izquierda → Raíz → Derecha

13. ¿En qué momento se muestra la raíz en el recorrido posorden?

Izquierda → Derecha → Raíz

14. ¿Por qué el recorrido inorden muestra los valores ordenados?

Por su estructura, todo nodo a la izquierda contiene valores menores, y todo nodo a la derecha contiene valores mayores que el nodo

15. ¿Qué sucede si los valores se insertan de menor a mayor?

En lugar de ramificarse, cada nodo nuevo se coloca siempre como el hijo derecho del nodo anterior

16. ¿Cuál es la diferencia entre el while del menú y las llamadas utilizadas dentro de la clase BinaryTree?

Que la función del while es permitir que el usuario realice varias operaciones sin tener que ejecutar nuevamente el programa.

17. ¿Qué parte del programa evita que se ingresen letras cuando se solicita un número?

```
if option == "1":
    try:
        value = int(input("Ingresa el valor que deseas insertar: "))
        tree.insertar(value)
        print("Operación realizada.")
    except ValueError:
        print("Debes ingresar un número entero.")
```

18. ¿Por qué la forma del árbol depende del orden de inserción?

Porque cada nuevo nodo se coloca comparándolo con los ya existentes y se compara hasta encontrar su posición ideal.

19. ¿Por qué desde el menú no se debe acceder directamente a tree.root?

Porque la clase BinaryTree es responsable de iniciar cada recorrido desde la raíz.

20. ¿Qué función cumplen los métodos _preorden, _inorden y _posorden?

Realizan las operaciones de acuerdo su orden, por ejemplo:

19. Recorre el subárbol izquierdo.

- 20. Muestra el valor del nodo actual.
- 21. Recorre el subárbol derecho.

ó

- 22. Recorre el subárbol izquierdo.
- 23. Recorre el subárbol derecho.
- 24. Muestra el valor del nodo actual.

ó

- 16. Muestra el valor del nodo actual.
- 17. Recorre el subárbol izquierdo.
- 18. Recorre el subárbol derecho.